Studio Nx

P1-09 비밀번호 변경 · 탈퇴 — 함수 설명 합본 (44~49)

2026-10-07

목차

1. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/44_user_service_check_current_password.md
2. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/45_user_service_change_password.md
3. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/46_user_service_withdraw.md
4. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/47_users_route_change_password.md
5. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/48_users_route_withdraw.md
6. ai-notes/backend(fastAPI_Python)/P1-09 - 비밀번호 변경·탈퇴/49_retention_purge_withdrawn_accounts.md

Backend(FastAPI/Python) 44 — `check_current_password()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경 · 탈퇴 — 비밀번호 재확인 · PRD 01 FR-107 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	43 GET /profile-images/{name} · 45 change_password()

1. 하는 일

이미 로그인한 회원에게 비밀번호를 한 번 더 확인한다. 맞으면 아무것도 하지 않고(`None`), 틀리면 400.

저장된 비밀번호 `Abcdef1!` 에 넣은 값	결과
`Abcdef1!`	통과 (`None`)
`Abcdef1?` · `abcdef1!` · `" Abcdef1!"` · 빈 글자	400 `WRONG_PASSWORD` "Password is incorrect."
100자 (bcrypt 한도 72바이트 넘음)	400 (같음 — 500 아님)
해시가 `"!"` (보관 기간 정리 뒤)	어떤 값이든 400

2. 언제 쓰나

쓰는 곳	왜 다시 묻나
45 `change_password`	"지금 비밀번호" 칸 — 잠깐 자리를 비운 사이 다른 사람이 비밀번호를 바꾸지 못하게
46 `withdraw`	탈퇴 창의 비밀번호 (FIX-01 `promptDialog`) — 되돌릴 수 없는 일이라서

로그인(29 `authenticate`)과 다른 점

	29 로그인	44 재확인
회원은	아직 모름 (이메일로 찾음)	이미 앎 (토큰의 회원)
틀리면	401 `INVALID_CREDENTIALS`	**400** `WRONG_PASSWORD`
시간 맞추기(가짜 해시)	함 — 없는 이메일인지 숨기려고	필요 없음 — 숨길 것이 없음

3. 코드

import (파일 맨 위)

```
from app.core.errors import AppError, ErrorCode
from app.core.security import verify_password
```

`verify_password` **줄을 이번에 추가했다** (나머지 import 는 P1-08 그대로).

함수

```
def check_current_password(user: User, password: str) -> None:
    if not verify_password(password, user.password_hash):
        raise AppError(ErrorCode.WRONG_PASSWORD, 400, "Password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import verify_password`	P1-04 (16) — bcrypt 로 비교, 오류는 `False`

식	뜻
`-> None`	돌려주는 값 없음. "통과하면 조용히 끝나고, 아니면 오류" 인 검사 함수
`if not …: raise …`	틀릴 때만 멈춤

4. 흐름

1. 넣은 비밀번호를 저장된 해시와 비교한다.
2. 틀리면 400. 맞으면 끝.

5. 왜 이렇게

선택	이유
400 (401 아님)	화면의 `api.js` (P1-02 02)는 401 을 받으면 "토큰이 만료됐다" 고 보고 재발급을 시도하고, 그래도 401 이면 로그인 화면으로 보낸다. 비밀번호를 잘못 쳤을 뿐인데 **로그아웃되면 안 된다**
별도 코드 `WRONG_PASSWORD`	화면이 "Current password is incorrect." 처럼 그 칸에 문구를 붙일 수 있게 (P1-10)
함수로 분리 (두 줄이지만)	45 · 46 이 같은 규칙을 쓴다. 문구 · 코드가 한 곳에 있다
DB 를 보지 않음	06 `CurrentUser` 가 이미 최신 회원을 읽어 왔다

알아 둘 점 — 여러 번 틀리기: 로그인한 사람이 이 창에서 비밀번호를 계속 바꿔 치며 맞혀 볼 수는 있다. 하지만 이미 그 계정으로 로그인한 상태라 얻을 것이 적다. 횟수 제한은 로그인 반복 제한과 함께 나중에 검토한다.

6. 시험 (`tests/test_user_service.py` 에 7개 추가 · 모두 63개)

시험	확인한 것
맞는 비밀번호	`None`
틀림 5가지	한 글자 · 대소문자 · 앞 공백 · 빈 글자 · 100자 → 400 `WRONG_PASSWORD` "Password is incorrect."
해시 `"!"`	400

전체 시험 321개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 45 — `change_password()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경 · 탈퇴 — 비밀번호 변경 · PRD 01 FR-107
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	44 `check_current_password()` · 46 `withdraw()`

1. 하는 일

지금 비밀번호를 확인하고, 새 비밀번호가 규칙에 맞으면 새 해시로 바꿔 저장한다.

지금 `Abcdef1!` → 보낸 값 (지금 / 새)	결과
`Abcdef1!` / `Newpass9#`	저장 — 이후 `Newpass9#` 로만 로그인
`Abcdef1?` / `Newpass9#`	400 `WRONG_PASSWORD` "Password is incorrect." (44)
`Abcdef1!` / `abcdefgh`	422 "Check the password requirements." · `details: {"new_password": ["upper", "number", "special"]}`
`Abcdef1!` / `Ab1!`	422 · `["length"]`
`Abcdef1!` / `Abcdef1!` 한글	422 · `["allowed"]`
`Abcdef1!` / `Abcdef1!` (같음)	422 **"Use a different password."** · `details: {"new_password": "same_as_current"}`
다른 회원이 `Newpass9#` 를 씀	상관없이 저장

오류일 때는 DB 의 해시가 그대로다.

2. 언제 쓰나

쓰는 곳	
`PUT /users/me/password` (47)	userInfo.html 비밀번호 칸 3개(지금 · 새 · 확인) → Change password (P1-10)

사용자 결정 (2026-10-07)

새 비밀번호가 지금 비밀번호와 같으면 막는다. 템플릿에는 "(선택)" 이었다.

- 이것은 **같은 회원 안**의 비교다. 다른 회원과 비밀번호가 겹치는지는 검사하지 않는다 — 겹쳐도 문제가 없고, 검사하면 "누군가 이 비밀번호를 쓴다" 는 단서가 된다. 해시마다 소금(salt)이 달라 비교할 수도 없다.
- 예전에 쓰던 비밀번호들까지 막는 것(이력)은 범위 밖.

3. 코드

import (파일 맨 위)

```
from app.core.security import check_password_rules, hash_password, verify_password
```

`check_password_rules` · `hash_password` 를 이번에 추가했다 (44 의 `verify_password` 줄에).

함수

```
def change_password(db: Session, user: User, current_password: str, new_password: str) -> None:
    check_current_password(user, current_password)
    failed = check_password_rules(new_password)
    if failed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Check the password requirements.",
            details={"new_password": failed},
        )
    if verify_password(new_password, user.password_hash):
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Use a different password.",
            details={"new_password": "same_as_current"},
        )
    user.password_hash = hash_password(new_password)
    db.commit()
```

문법 풀이

import 문법	뜻
`check_password_rules` · `hash_password`	P1-04 (14 · 15) — 가입과 같은 규칙 · 해시

식	뜻
`if failed:`	빈 목록이면 거짓 — 실패한 조건이 하나라도 있으면 참
`verify_password(new_password, user.password_hash)`	새 비밀번호를 **지금 해시**에 대 봄 — 맞으면 지금 비밀번호와 같다는 뜻
`-> None`	돌려줄 것 없음. 라우터는 204 (본문 없음)

4. 흐름

1. 지금 비밀번호 확인 → 틀리면 400.
2. 새 비밀번호 규칙 → 틀리면 422 + 실패한 조건.
3. 지금과 같으면 422.
4. 새 해시로 저장.

5. 왜 이렇게

선택	이유
지금 비밀번호를 **먼저**	남이 내 자리에서 새 비밀번호 규칙을 시험해 볼 수도 없게, 본인 확인이 가장 먼저

선택	이유
규칙 검사를 같음 검사보다 먼저	지금 비밀번호는 규칙을 통과했던 값이라 순서가 결과를 바꾸지 않는다. DB 를 보지 않는 가벼운 검사를 앞에 (P1-04 와 같은 원칙)
같음 비교는 `verify_password`	지금 비밀번호 원문은 서버에 없다 — 해시에 대 봐야 안다. 지금 비밀번호 칸 값과 글자 비교(`current == new`)도 되지만, 해시와 비교해야 "실제로 저장된 것과 같음" 이 확실
`details` 칸 이름 `new_password`	화면(P1-10)이 그 칸 아래에 문구를 붙일 수 있게 — 요청 본문의 칸 이름과 같음
`refresh` 하지 않음	돌려줄 정보가 없다 (응답 204)

알아 둘 점 — 다른 기기: 비밀번호를 바꿔도 다른 브라우저에 이미 있는 리프레시 쿠키(최대 3일)는 그대로 동작한다. 리프레시 토큰을 DB 에 두지 않는 설계라서다 (P1-05 34 §5 와 같은 한계). 필요해지면 "이 시각 이전에 만든 토큰은 무효" 를 회원 행에 두는 방법이 있다 (토큰의 `iat`, P1-05 26).

6. 시험 (`tests/test_user_service.py` 에 7개 추가 · 모두 70개, 실제 MySQL)

시험	확인한 것
바꾸기	DB 해시가 바뀜, 새 비밀번호는 맞고 옛 비밀번호는 틀림
지금 비밀번호 틀림	400 `WRONG_PASSWORD`, 해시 그대로
규칙 3가지	약함 · 짧음 · 한글 공백 → 422, `details.new_password` 가 실패 목록
지금과 같음	422 "Use a different password.", `same_as_current`, 해시 그대로
다른 회원과 같은 비밀번호	저장됨

전체 시험 328개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 46 — `withdraw()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경 · 탈퇴 — 회원 탈퇴(소프트 삭제) · PRD 01 FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	45 `change_password()` · 47 `PUT /users/me/password`

1. 하는 일

비밀번호를 확인하고, 회원을 **탈퇴 상태**로 바꾼다. 행은 지우지 않는다(소프트 삭제). 프로필 사진은 지운다.

바뀌는 것	바뀌지 않는 것
`status` → `DELETED`	이메일 · 닉네임 (3일 동안 재가입을 막는 데 씀)
`deleted_at` → 지금 (UTC)	회원 행 자체
`profile_image_path` → `None`, 사진 파일 삭제	내가 OWNER 인 프로젝트

상황	결과
비밀번호 맞음	위처럼 바뀜
비밀번호 틀림	400 `WRONG_PASSWORD` — 아무것도 바뀌지 않음, 사진도 그대로
저장 실패	오류 그대로 — 사진 파일도 남음

탈퇴 뒤에 일어나는 일 (다른 함수들이 이미 함)

무엇	결과	막는 곳
로그인	401 (이유는 숨김)	29 `authenticate`
남은 Access Token (최대 30분)	401	06 `get_current_user`
자동 로그인 쿠키로 재발급	401	30 `get_active_user`
같은 이메일 · 닉네임으로 3일 안 가입	409 `REJOIN_TOO_SOON`	19 · 20 (P1-04)
사진 주소	404	40
3일 뒤	이메일 · 닉네임 · 남은 개인 정보 비움	49 보관 기간 정리 (다음 다음)

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me` (48)	본문 `{"password": "…"}` → 이 함수 → 리프레시 쿠키 삭제 → 204
화면	userInfo.html Delete account → FIX-01 `promptDialog` (비밀번호만, 체크박스 없음) — P1-10

3. 코드

import (파일 맨 위)

```
import re
from datetime import UTC, datetime
from pathlib import Path, PurePosixPath
…
```

`from datetime import UTC, datetime` 을 이번에 추가했다. `UserStatus` 는 40 에서 이미 넣었다.

함수

```
def withdraw(db: Session, user: User, password: str) -> None:
    check_current_password(user, password)
    user.status = UserStatus.DELETED
    user.deleted_at = datetime.now(UTC).replace(tzinfo=None)
    delete_profile_image(db, user)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime`	지금 시각 (UTC 기준)

식	뜻
`datetime.now(UTC).replace(tzinfo=None)`	UTC 지금 시각에서 시간대 표시를 뗌 — DB `DATETIME` 칸은 시간대를 저장하지 않고 늘 UTC 로 쓴다 (P1-04 와 같음)
`user.status = …` · `user.deleted_at = …`	바꾸기만 — 아직 DB 에 저장되지 않음
`delete_profile_image(db, user)`	39 — 사진 경로를 비우고 **`commit`** → 위의 두 줄도 이 `commit` 으로 함께 저장. 저장 뒤 파일 삭제

4. 흐름

1. 비밀번호 확인 → 틀리면 400 (아무것도 바뀌지 않음).
2. 상태 · 탈퇴 시각을 바꾼다 (아직 저장 전).
3. 39 를 부른다 → 사진 경로를 비우고 한 번에 저장 → 사진 파일 삭제.

5. 왜 이렇게

선택	이유
39 를 **마지막에** 불러 저장을 한 번으로	템플릿 요구("commit 은 마지막에 한 번"). 상태만 저장되고 사진은 남는(또는 반대) 반쪽 탈퇴가 생기지 않는다. 39 를 만들 때 이 쓰임을 정해 두었다 (39 §5)
이메일 · 닉네임을 바로 비우지 않음	2026-10-03 결정 — 탈퇴하고 같은 이메일로 바로 다시 가입해 제재 · 기록을 피하는 것을 3일 동안 막는다
프로젝트를 남김	2026-10-03 결정 — 함께 작업한 사람의 곡이 사라지면 안 된다. 다른 사람에게는 'Deleted user' 로 보임 (Phase 6, 추적 방지 ①)
리프레시 쿠키는 라우터에서	쿠키는 **응답**에 붙는 것이라 서비스(DB 일)와 나눴다. 쿠키를 못 지워도 30 이 재발급을 막는다
사진 파일은 바로 지움	3일을 기다릴 이유가 없다 — 재가입을 막는 데 쓰이지 않는다

6. 시험 (`tests/test_user_service.py` 에 4개 추가 · 모두 74개, 실제 MySQL + 임시 폴더)

시험	확인한 것
탈퇴	DB 에 `DELETED` · 지금 시각, 이메일 · 닉네임 그대로, 사진 경로 `None` · 파일 삭제, OWNER 프로젝트 그대로
비밀번호 틀림	400, 상태 `ACTIVE`, 사진 파일 그대로
탈퇴 뒤	로그인 401 · 재발급 401 · 같은 이메일 · 닉네임 `REJOIN_TOO_SOON`
저장 실패	오류 그대로, 사진 파일 남음

전체 시험 332개 통과, `ruff` 검사 통과. (P1-08 사이트 시험이 남긴 빈 `storage/profiles/` 폴더를 지웠다 — 이제 `.gitkeep` 만)

Backend(FastAPI/Python) 47 — `PUT /api/v1/users/me/password`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `change_password()`
작업	P1-09 비밀번호 변경 · 탈퇴 — 비밀번호 변경 · PRD 01 FR-107
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	46 `withdraw()` · 48 `DELETE /users/me`

1. 하는 일

요청 본문 (지금 `Abcdef1!`)	응답
`{"current_password": "Abcdef1!", "new_password": "Newpass9#"}`	**204** (본문 없음) — 이후 `Newpass9#` 로만 로그인
지금 비밀번호 틀림	**400** `WRONG_PASSWORD` "Password is incorrect."
새 비밀번호 `abcdefgh`	422 "Check the password requirements." · `details: {"new_password": ["upper", "number", "special"]}`
새 비밀번호 = 지금	422 "Use a different password." · `details: {"new_password": "same_as_current"}`
`current_password` 빈 글자	422 (스키마) · `details` 에 `current_password`
토큰 없음	401

2. 언제 쓰나

화면 (userInfo.html, P1-10)	
Current password · New password · Confirm → Change password	`api.put('/users/me/password', { current_password, new_password })`. 확인 칸 비교는 화면이 함 (가입 16 과 같은 방식)
400 `WRONG_PASSWORD`	지금 비밀번호 칸 아래 "Current password is incorrect."
422 `details.new_password`	새 비밀번호 칸 아래

3. 코드

import (파일 맨 위 — 41 과 같음, 추가 없음)

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.put(
    "/me/password",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={400: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def change_password(data: PasswordChangeRequest, user: CurrentUser, db: Db) -> None:
    user_service.change_password(db, user, data.current_password, data.new_password)
```

문법 풀이

식	뜻
`@router.put(…)`	**PUT** — "이 값으로 바꿔 놓아라". 비밀번호는 통째로 새 값이 되므로 PATCH(일부 수정)가 아니라 PUT
`status_code=status.HTTP_204_NO_CONTENT`	성공하면 204 — 돌려줄 내용이 없음
`data: PasswordChangeRequest`	템플릿 스키마 — 두 칸 모두 1~100자 (빈 칸이면 422)
`-> None`	본문 없음

4. 흐름

토큰 확인(06) → 본문 검사(스키마) → 45 (지금 비밀번호 → 규칙 → 같음 → 저장) → 204.

5. 왜 이렇게

선택	이유
라우터는 한 줄	규칙은 44 · 45 에서 시험을 마쳤다
204 (새 토큰을 주지 않음)	Access Token 은 회원 번호만 담는다 — 비밀번호가 바뀌어도 그대로 쓸 수 있다. 다시 로그인시키지 않는다
틀리면 400	44 §5 — 401 이면 화면이 로그아웃시킴
새 비밀번호 확인 칸은 서버에 보내지 않음	두 칸이 같은지는 화면이 확인한다 (가입과 같음). 서버는 같은 값을 두 번 받아도 할 일이 없다

6. 시험 (`tests/test_user_routes.py` 에 6개 추가 · 모두 32개, 실제 앱 + MySQL)

시험	확인한 것
바꾸기	204 · 본문 없음 → **실제 로그인 API** 로 새 비밀번호 200, 옛 비밀번호 401
지금 비밀번호 틀림	400 `WRONG_PASSWORD` "Password is incorrect."
규칙 · 지금과 같음	422 + 문구 + `details.new_password`
지금 비밀번호 빈 칸	422 · `details` 에 `current_password`
토큰 없음	401

전체 시험 338개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 48 — `DELETE /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `withdraw()`
작업	P1-09 비밀번호 변경 · 탈퇴 — 회원 탈퇴 · PRD 01 FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	47 `PUT /users/me/password` · 49 `purge_withdrawn_accounts()`

1. 하는 일

요청	응답
본문 `{"password": "Abcdef1!"}` (맞음)	**204** + `Set-Cookie: snx_refresh=""; Max-Age=0; Path=/api/v1/auth; HttpOnly; SameSite=lax` (쿠키 삭제)
비밀번호 틀림	400 `WRONG_PASSWORD` — 쿠키 그대로, 계속 로그인 상태
본문 없음 · `{}` · 빈 글자	422 — 탈퇴하지 않음
토큰 없음	401

204 뒤 (실제 앱으로 확인)

무엇	결과
남은 Access Token 으로 `GET /users/me`	401
`POST /auth/refresh`	401 — 쿠키가 지워짐
같은 이메일 · 비밀번호로 로그인	401
사진 주소	404

2. 언제 쓰나

화면 (userInfo.html, P1-10)	
Delete account → FIX-01 `promptDialog` (비밀번호, `check` 안에서 이 API)	`api.delete('/users/me', { body: { password } })`
204	`clearAccessToken()` → `home.html`
400 `WRONG_PASSWORD`	창 안 "Password is incorrect." — 창이 닫히지 않음 (21 `check`)

함께 고친 것 — 화면 쪽 메모

`api.js`(P1-02)의 `api.delete(path, options)` 는 두 번째 인자가 **옵션**이다. 본문은 `{ body: … }` 로 넣어야 한다. `userinfo.js` 템플릿 주석과 FIX-01 21 문서의 예시가 `api.delete('/users/me', { password })` 로 되어 있어 — 그대로 쓰면 본문 없이 가서 422 — 둘 다 `{ body: { password } }` 로 고쳤다.

3. 코드

import (파일 맨 위)

```
from app.api.deps import CurrentUser, Db
from app.api.v1.auth import clear_refresh_cookie
```

`clear_refresh_cookie` 줄을 이번에 추가했다 (P1-05 31).

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.delete(
    "/me",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={400: {"model": ErrorResponse}},
)
def withdraw(data: WithdrawRequest, response: Response, user: CurrentUser, db: Db) -> None:
    user_service.withdraw(db, user, data.password)
    clear_refresh_cookie(response)
```

문법 풀이

import 문법	뜻
`from app.api.v1.auth import clear_refresh_cookie`	로그아웃(34)이 쓰는 쿠키 지우기 — 심을 때와 같은 속성으로 지워야 브라우저가 같은 쿠키로 알아봄

식	뜻
`@router.delete("/me", …)`	`DELETE /api/v1/users/me` — 같은 주소의 `GET` · `PATCH` (41) 와 방법(메서드)만 다름
`data: WithdrawRequest`	DELETE 인데도 **본문**을 받음 (HTTP 가 허용함). 비밀번호를 주소에 넣으면 서버 기록 · 브라우저 기록에 남기 때문
`response: Response`	FastAPI 가 넣어 주는 응답 객체 — 여기에 쿠키 삭제를 붙이면 204 응답에 실려 감

4. 흐름

토큰 확인(06) → 본문 검사 → 46 (비밀번호 → 탈퇴 상태 · 사진 · 저장) → 쿠키 삭제 → 204. 46 이 오류(400)를 던지면 쿠키 삭제 줄까지 오지 않는다 → 계속 로그인 상태.

5. 왜 이렇게

선택	이유
쿠키 삭제는 **탈퇴 성공 뒤에만**	비밀번호를 틀렸는데 로그아웃되면 안 된다 (시험으로 확인)
쿠키를 지워도 30 이 이중으로 막음	다른 브라우저 · 기기에 남은 쿠키는 이 응답이 지울 수 없다 — 그 쿠키로 재발급을 시도하면 30 `get_active_user` 가 `DELETED` 를 보고 401
비밀번호는 본문으로	주소(`?password=`)는 서버 접속 기록 · 브라우저 방문 기록에 남는다
204	탈퇴한 사람에게 돌려줄 정보가 없다

6. 시험 (`tests/test_user_routes.py` 에 6개 추가 · 모두 38개, 실제 앱 + MySQL + 임시 폴더)

시험	확인한 것
탈퇴 (실제 로그인 → 사진 → 재발급 200 → 탈퇴)	204 · 본문 없음 · 쿠키 삭제 헤더(`Max-Age=0`, `Path=/api/v1/auth`) → 남은 토큰 401 · 재발급 401 · 다시 로그인 401 · 사진 404 · DB `DELETED`
비밀번호 틀림	400 `WRONG_PASSWORD`, 쿠키 삭제 헤더 없음, 내 정보 200 · 재발급 200 그대로
본문 없음 · `{}` · 빈 글자	422, DB `ACTIVE` 그대로
토큰 없음	401

전체 시험 344개 통과, `ruff` 검사 통과.

Backend(FastAPI/Python) 49 — `purge_withdrawn_accounts()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-09 비밀번호 변경 · 탈퇴 — 탈퇴 3일 뒤 개인 정보 정리 · PRD 01 FR-108 · 개인정보 처리방침 §4
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	48 `DELETE /users/me` · (P1-09 마지막)

1. 하는 일

하루 한 번 돌리는 **정리 작업**. 탈퇴한 지 3일이 지난 계정에서 개인 정보를 비운다. 처리방침(privacy.html §4)에 적은 "탈퇴하면 3일 보관 뒤 비움" 약속을 실제로 지키는 곳.

칸	정리 전	정리 후
`email`	`old.gone@example.com`	`deleted+85@withdrawn.invalid`
`nickname`	`old_gone`	`deleted_user_0000000085`
`password_hash`	`$2b$12$…`	`!` (어떤 비밀번호와도 맞지 않음)
`last_login_at`	마지막 로그인 시각	`None`
`profile_image_path` · 사진 파일	(남아 있으면)	`None` · 파일 삭제
`status` · `deleted_at` · 행 · 프로젝트		**그대로**

계정	결과
탈퇴 4일 · 정확히 3일	정리
탈퇴 2일	그대로 — 3일 동안은 재가입을 막는 데 이메일 · 닉네임을 씀
활동 회원	그대로
이미 다 비운 계정	건너뜀 (세지도 않음)
누가 그 이메일로 가입해 이메일 · 닉네임만 비켜 둔 계정	남은 비밀번호 해시 등을 마저 비움

돌려주는 값 = 이번에 비운 계정 수.

2. 언제 쓰나

실행	언제
`python -m app.workers.retention` 의 `main()`	하루 한 번 (운영체제 예약 작업 — 배포할 때 정함). **`main()` 은 P1-12** — 문의 정리(`purge_old_inquiries`)와 함께 부르므로 그 함수를 만들 때 구현

탈퇴 회원의 시간표

시점	상태	하는 곳
탈퇴 순간	`DELETED`, 사진 삭제, 로그인 · 재발급 차단	46 · 48
~ 3일	같은 이메일 · 닉네임으로 가입 불가 (409 `REJOIN_TOO_SOON`)	P1-04

시점	상태	하는 곳
3일 뒤 누가 그 값을 쓰려 함	이메일 · 닉네임만 바로 비킴	P1-04 `release_withdrawn`
3일 뒤 첫 정리 작업	**나머지까지 모두 비움**	**49**

추적 방지 (2026-10-07 결정 ②)

비켜 둔 값에는 회원 번호만 남고, 원래 이메일 · 닉네임 · 비밀번호 해시 · 마지막 로그인 · 사진이 남지 않는다. ① 다른 사람에게 보이는 곳은 'Deleted user', ③ 번호로 조회하는 공개 API 없음(P1-08 사진 주소 포함)과 함께 쓴다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`datetime` 줄 · `Session` 줄 말고는 모두 이번에 추가했다.

상수

```
# 다 비운 계정의 비밀번호 칸 값 — bcrypt 해시 모양이 아니라 verify_password 가 늘 False
# (칸이 NOT NULL 이라 빈칸 대신)
WIPED_PASSWORD_HASH = "!"
```

함수

```
def purge_withdrawn_accounts(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=REJOIN_WAIT_DAYS)
    users = db.scalars(
        select(User).where(
            User.status == UserStatus.DELETED,
            User.deleted_at <= cutoff,
            or_(
                ~User.email.startswith("deleted+", autoescape=True),
                ~User.nickname.startswith("deleted_user_", autoescape=True),
                User.password_hash != WIPED_PASSWORD_HASH,
                User.last_login_at.is_not(None),
                User.profile_image_path.is_not(None),
            ),
        )
    ).all()
```

```
    photos = []
    for user in users:
        release_withdrawn(db, user)
        if user.profile_image_path:
            photos.append(user.profile_image_path)
        user.password_hash = WIPED_PASSWORD_HASH
        user.last_login_at = None
        user.profile_image_path = None
    db.commit()

    storage_dir = get_settings().storage_dir
    for relative in photos:
        (storage_dir / relative).unlink(missing_ok=True)
    return len(users)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import or_, select`	`or_(A, B, …)` = SQL 의 `A OR B OR …`
`from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn`	3일 · 비켜 두기 — 가입 쪽(P1-04)과 **같은 값 · 같은 함수**를 씀

식	뜻
`now.astimezone(UTC).replace(tzinfo=None)`	한국 시각처럼 시간대가 붙은 값을 UTC 로 바꾸고 표시를 뗌 — DB 는 UTC
`User.email.startswith("deleted+", autoescape=True)`	SQL `LIKE 'deleted+%'`. `autoescape` 는 값 안의 `_` · `%` 를 글자 그대로 다룸 (`LIKE` 에서 `_` 는 "아무 한 글자")
`~조건`	NOT
`.is_not(None)`	`IS NOT NULL`
`db.scalars(…).all()`	조건에 맞는 `User` 들을 목록으로

4. 흐름

1. 기준 시각 = 지금 − 3일 (UTC).
2. 탈퇴했고 기준보다 오래됐으며 **아직 남은 것이 하나라도 있는** 계정을 찾는다.
3. 각 계정: 이메일 · 닉네임 비키기 → 사진 경로 기억 → 비밀번호 · 마지막 로그인 · 사진 경로 비우기.
4. 한 번에 저장.
5. 저장 뒤 사진 파일 삭제 (이미 없으면 넘어감).
6. 비운 수를 돌려준다.

5. 왜 이렇게

선택	이유
"남은 것이 하나라도 있는" 행만	매일 돌려도 같은 행을 다시 세지 않는다. 가입 때 이메일 · 닉네임만 비켜 둔 행도 빠뜨리지 않는다 (템플릿은 "이메일이 `deleted+` 가 아닌 행" 만 — 그러면 그런 행의 비밀번호 해시가 영영 남는다)
`password_hash = "!"`	칸이 `NOT NULL` 이라 비울 수 없다. `"!"` 는 bcrypt 해시 모양이 아니어서 어떤 비밀번호로도 로그인할 수 없다 (44 시험에서 확인)

선택	이유
`release_withdrawn` 을 그대로 씀	비켜 두는 값이 가입 쪽과 늘 같다. 여러 번 불려도 같은 결과
`deleted_at` · 행을 남김	프로젝트(owner_id)가 이 행을 가리킨다. 탈퇴 시각은 기록으로 필요
사진 파일은 저장 **뒤** 삭제	38 · 39 와 같은 순서 — 저장이 실패하면 파일도 남는다
`now` 를 받음 (안에서 시각을 재지 않음)	시험에서 시각을 정해 넣을 수 있다 ("정확히 3일" 같은 경계)
시간대가 있는 `now` 도 받음	실행하는 곳이 한국 시각을 넘겨도 틀리지 않게

알아 둘 점 — 색인: `status` · `deleted_at` 조건으로 회원 표를 훑는다. 하루 한 번이라 회원이 많아도 부담이 적다.

6. 시험 (`tests/test_retention.py` 새 파일 · 8개, 실제 MySQL + 임시 폴더)

시험	확인한 것
탈퇴 4일 (사진 · 로그인 기록 · 프로젝트 있음)	1 · 이메일 · 닉네임 비킴 · 해시 `!` · 마지막 로그인 · 사진 경로 `None` · 파일 삭제 · 상태 · 탈퇴 시각 · 프로젝트 그대로
탈퇴 2일 · 활동 회원	0 · 그대로
정확히 3일	정리
두 번 돌리기	1 → 0
가입 때 이메일 · 닉네임만 비켜 둔 행	1 · 해시 · 로그인 기록도 비움
사진 파일이 이미 없음	오류 없이 정리
한국 시각 21:00 (= UTC 12:00) 로 실행	정확히 3일로 계산해 정리
정리 뒤	옛 비밀번호로 로그인 401, 옛 이메일 · 닉네임으로 가입 가능

전체 시험 352개 통과, `ruff` 검사 통과.